在VScode中构建Verilog环境以及自动化生成，运行项目文件的方法

基础工作

安装扩展

1. **Verilog-HDL/SystemVerilog/Bluespec SystemVerilog**
   - 作者：`mshr-h`
   - 提供语法高亮、代码补全、Linting
2. **TerosHDL**（强烈推荐）
   - 作者：`TerosHDL`
   - 功能：模块层次图、文档生成、状态机可视化、模板生成
3. **WaveTrace**
   - 在 VS Code 内查看波形（`.vcd` 文件）

项目结构示例

在 VS Code 中创建一个新文件夹作为项目根目录，建议结构：

```
verilog_learning/
├── rtl/              # 存放设计代码（.v）
│   └── test.v
├── tb/               # 存放测试平台
│   └── tb_test.v
├── sim/              # 存放仿真生成的文件
└── .vscode/          # VS Code 配置
    └── tasks.json    # 自动化任务配置
```

配置自动化任务

在项目根目录创建 `.vscode/tasks.json`，这样就能**一键编译+仿真**。

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "iverilog: compile",
            "type": "shell",
            "command": "iverilog",
            "args": [
                "-g2012",
                "-o",
                "sim/${fileBasenameNoExtension}.vvp",
                "-I",
                "rtl",
                "rtl/*.v",
                "${file}"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "problemMatcher": [],
            "presentation": {
                "reveal": "always",
                "panel": "new"
            }
        },
        {
            "label": "vvp: simulate",
            "type": "shell",
            "command": "vvp",
            "args": [
                "sim/${fileBasenameNoExtension}.vvp"
            ],
            "dependsOn": "iverilog: compile",
```

```
            "problemMatcher": [],
            "presentation": {
                "reveal": "always",
                "panel": "shared"
            }
        },
        {
            "label": "gtkwave: view waveform",
            "type": "shell",
            "command": "gtkwave",
            "args": [
                "sim/${fileBasenameNoExtension}.vcd"
            ],
            "problemMatcher": []
        },
        {
            "label": "Run Simulation (compile + simulate + wave)",
            "dependsOn": [
                "vvp: simulate"
            ],
            "problemMatcher": []
        }
    ]
}
```

运行仿真

方法 1：使用快捷键（推荐）

1. 在 VS Code 中打开 `tb/tb_test.v`
2. 按 `Ctrl+Shift+B` （或 `Cmd+Shift+B` ）
3. 选择 **"Run Simulation (compile + simulate + wave)"**
4. 查看终端输出的真值表
5. 手动运行查看波形：

```
gtkwave sim/test.vcd
```

方法 2：在终端手动运行

在项目根目录打开终端：

```
# 1. 创建 sim 目录
mkdir -p sim

# 2. 编译
iverilog -g2012 -o sim/tb_test.vvp rtl/test.v tb/tb_test.v

# 3. 运行仿真
vvp sim/test.vvp

# 4. 查看波形
gtkwave sim/test.vcd
```

查看波形（GTKWave）

运行 `gtkwave sim/test.vcd` 后：

1. 左侧 **SST** 窗口点击 `test`
2. 选中 `a`，`b`，`y`
3. 点击 **"Append"** 或直接拖到右侧波形区
4. 你会看到：

```
a: ________    ________
           ____        ____
b: ____    ____    ____
       ____    ____    ____
y: ________________    ____
                   ____
```

完整的 Verilog 开发流程

```
编辑代码 (VS Code)
    ↓
编译 (iverilog)
    ↓
仿真 (vvp)
    ↓
查看波形 (gtkwave)
```

安装 Icarus Verilog（Windows）

方法1：使用安装包（推荐，最简单）

Step 1: 下载安装包

访问以下任一链接下载：

官方下载页面：

- http://bleyer.org/icarus/
- 选择最新版本，例如：`iverilog-v11-20211123-x64_setup.exe`

或者直接下载（备用镜像）：

- https://github.com/steveicarus/iverilog/releases

Step 2: 安装

1. 运行安装包
2. **重要**：安装过程中勾选 **"Add executable folder(s) to the user PATH"**

   ![安装时的界面示意]

   ```
   ☑ Add executable folder(s) to the user PATH
   ```

3. 默认安装路径通常是：`C:\iverilog`
4. 一路 "Next" 完成安装

Step 3: 验证安装

1. **关闭当前所有 PowerShell/CMD/VS Code 窗口**
2. 重新打开 PowerShell 或 VS Code 终端
3. 输入：

```
iverilog -v
```

如果看到类似输出，说明安装成功：

```
Icarus Verilog version 11.0 (stable) ()
...
```

方法2：手动添加到 PATH（如果方法1没自动添加）

如果安装后仍然提示找不到命令：

Step 1: 找到安装目录

默认路径：`C:\iverilog\bin`

Step 2: 添加到系统 PATH

1. 按 `Win + X`，选择 **"系统"**
2. 点击右侧 **"高级系统设置"**
3. 点击 **"环境变量"**
4. 在 **"用户变量"** 或 **"系统变量"** 中找到 `Path`
5. 点击 **"编辑"** → **"新建"**
6. 添加：`C:\iverilog\bin`（或你的实际安装路径）
7. 点击 **"确定"** 保存

Step 3: 重启 VS Code

必须完全关闭 VS Code 后重新打开，环境变量才会生效。

Step 4: 再次验证

```
iverilog -v
```

自动化项目生成脚本

项目生成脚本

`create_verilog_project.ps1`

把这个文件保存在打开文件的根目录（比如 `projects`），以后每次用它创建新项目。

```
# ========================================
# Verilog 项目自动生成脚本
# 作者: Stomatra
# 功能: 自动创建项目目录、模板文件、VS Code 配置
# 使用方法: .\create_verilog_project.ps1 -ProjectName "my_project" -ModuleName
"counter"
# ========================================

param(
    [Parameter(Mandatory=$true, HelpMessage="项目名称（文件夹名）")]
    [string]$ProjectName,

    [Parameter(Mandatory=$true, HelpMessage="模块名称（如 counter, adder）")]
    [string]$ModuleName,

    [Parameter(Mandatory=$false, HelpMessage="项目根目录（默认当前目录）")]
    [string]$BaseDir = "."
)

# ========================================
# 1. 创建项目目录
# ========================================
```

```
$projectPath = Join-Path $BaseDir $ProjectName

Write-Host "========================================" -ForegroundColor Cyan
Write-Host "  Verilog 项目生成器" -ForegroundColor Cyan
Write-Host "========================================" -ForegroundColor Cyan
Write-Host ""

if (Test-Path $projectPath) {
    Write-Host "⚠️  项目已存在: $projectPath" -ForegroundColor Yellow
    $overwrite = Read-Host "是否覆盖? (y/n)"
    if ($overwrite -ne "y") {
        Write-Host "❌ 取消创建" -ForegroundColor Red
        exit
    }
    Remove-Item -Path $projectPath -Recurse -Force
}

Write-Host "📁 创建项目目录..." -ForegroundColor Green
New-Item -ItemType Directory -Path $projectPath | Out-Null
New-Item -ItemType Directory -Path "$projectPath\rtl" | Out-Null
New-Item -ItemType Directory -Path "$projectPath\tb" | Out-Null
New-Item -ItemType Directory -Path "$projectPath\sim" | Out-Null
New-Item -ItemType Directory -Path "$projectPath\.vscode" | Out-Null

# ========================================
# 2. 生成 RTL 设计文件
# ========================================
$rtlFile = "$projectPath\rtl\$ModuleName.v"
Write-Host "📝 生成设计文件: rtl\$ModuleName.v" -ForegroundColor Green

$rtlContent = @"
// ========================================
// 模块名: $ModuleName
// 功能: (在这里描述你的模块功能)
// 作者: Stomatra
// 日期: $(Get-Date -Format "yyyy-MM-dd")
// ========================================
```

```
``timescale 1ns/1ps

module $ModuleName (
    input  wire clk,          // 时钟信号
    input  wire rst_n,        // 复位信号（低电平有效）
    input  wire [7:0] data_in,  // 8位输入数据
    output reg  [7:0] data_out  // 8位输出数据
);

    // ========================================
    // 内部信号声明
    // ========================================
    // 在这里声明内部信号（wire/reg）

    // ========================================
    // 组合逻辑
    // ========================================
    // 使用 assign 或 always @(*) 描述组合逻辑

    // ========================================
    // 时序逻辑
    // ========================================
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            // 异步复位
            data_out <= 8'h00;
        end else begin
            // 在这里实现你的时序逻辑
            data_out <= data_in;  // 示例：直通逻辑
        end
    end

endmodule
"@

Set-Content -Path $rtlFile -Value $rtlContent -Encoding UTF8

# ========================================
```

```
# 3. 生成 Testbench 文件
# ========================================
$tbFile = "$projectPath\tb\tb_$ModuleName.v"
Write-Host "📝 生成测试文件: tb\tb_$ModuleName.v" -ForegroundColor Green

$tbContent = @"
// ========================================
// 测试平台: tb_$ModuleName
// 功能: 测试 $ModuleName 模块
// 作者: Stomatra
// 日期: $(Get-Date -Format "yyyy-MM-dd")
// ========================================

``timescale 1ns/1ps

module tb_$ModuleName;

    // ========================================
    // 1. 参数定义
    // ========================================
    parameter CLK_PERIOD = 10;  // 时钟周期 10ns (100MHz)

    // ========================================
    // 2. 信号声明
    // ========================================
    reg         clk;
    reg         rst_n;
    reg  [7:0]  data_in;
    wire [7:0]  data_out;

    // ========================================
    // 3. 时钟生成
    // ========================================
    initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk;
    end
```

```
// ========================================
// 4. 例化待测设计 (DUT)
// ========================================
$ModuleName u_$ModuleName (
    .clk      (clk),
    .rst_n    (rst_n),
    .data_in  (data_in),
    .data_out (data_out)
);

// ========================================
// 5. 激励生成
// ========================================
initial begin
    // 初始化信号
    rst_n   = 0;
    data_in = 8'h00;

    // 打印测试开始信息
    `$display("========================================");
    `$display("  $ModuleName Testbench");
    `$display("  Start Time: %0t", `$time);
    `$display("========================================");

    // 复位
    #(CLK_PERIOD * 2);
    rst_n = 1;
    `$display("[%0t] Reset released", `$time);

    // 测试用例 1
    #(CLK_PERIOD);
    data_in = 8'hAA;
    `$display("[%0t] Test 1: data_in = 0x%h", `$time, data_in);

    // 测试用例 2
    #(CLK_PERIOD * 2);
    data_in = 8'h55;
    `$display("[%0t] Test 2: data_in = 0x%h", `$time, data_in);
```

```
        // 测试用例 3
        #(CLK_PERIOD * 2);
        data_in = 8'hFF;
        `$display("[%0t] Test 3: data_in = 0x%h", `$time, data_in);

        // 等待几个周期
        #(CLK_PERIOD * 5);

        // 测试结束
        `$display("========================================");
        `$display("  Simulation Finished!");
        `$display("  End Time: %0t", `$time);
        `$display("========================================");
        `$finish;
    end

    // ========================================
    // 6. 波形文件生成
    // ========================================
    initial begin
        `$dumpfile("sim/tb_$ModuleName.vcd");
        `$dumpvars(0, tb_$ModuleName);
    end

    // ========================================
    // 7. 监控输出（可选）
    // ========================================
    // initial begin
    //     `$monitor("Time=%0t clk=%b rst_n=%b data_in=%h data_out=%h",
    //         `$time, clk, rst_n, data_in, data_out);
    // end

endmodule
"@

Set-Content -Path $tbFile -Value $tbContent -Encoding UTF8
```

```
# ========================================
# 4. 生成 VS Code tasks.json
# ========================================
$tasksFile = "$projectPath\.vscode\tasks.json"
Write-Host "⚙️  生成 VS Code 配置: .vscode\tasks.json" -ForegroundColor Green

$tasksContent = @"
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Compile Verilog",
            "type": "shell",
            "command": "iverilog",
            "args": [
                "-g2012",
                "-Wall",
                "-o",
                "sim/`${fileBasenameNoExtension}.vvp",
                "-I", "rtl",
                "rtl/*.v",
                "`${file}"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "problemMatcher": [],
            "presentation": {
                "echo": true,
                "reveal": "always",
                "focus": false,
                "panel": "shared",
                "showReuseMessage": false,
                "clear": true
            }
        },
        {
```

```
            "label": "Run Simulation",
            "type": "shell",
            "command": "vvp",
            "args": [
                "sim/`${fileBasenameNoExtension}.vvp"
            ],
            "dependsOn": "Compile Verilog",
            "problemMatcher": [],
            "presentation": {
                "echo": true,
                "reveal": "always",
                "focus": true,
                "panel": "shared",
                "showReuseMessage": false
            }
        },
        {
            "label": "View Waveform",
            "type": "shell",
            "command": "gtkwave",
            "args": [
                "sim/`${fileBasenameNoExtension}.vcd"
            ],
            "problemMatcher": [],
            "presentation": {
                "reveal": "never"
            }
        },
        {
            "label": "Full Simulation Flow",
            "dependsOn": [
                "Run Simulation",
                "View Waveform"
            ],
            "dependsOrder": "sequence",
            "problemMatcher": []
        },
        {
```

```
            "label": "Clean Simulation Files",
            "type": "shell",
            "command": "Remove-Item",
            "args": [
                "-Path", "sim/*",
                "-Force",
                "-ErrorAction", "SilentlyContinue"
            ],
            "problemMatcher": [],
            "presentation": {
                "reveal": "always"
            }
        }
    ]
}
"@

Set-Content -Path $tasksFile -Value $tasksContent -Encoding UTF8

# ========================================
# 5. 生成 .gitignore
# ========================================
$gitignoreFile = "$projectPath\.gitignore"
Write-Host "📄 生成 .gitignore" -ForegroundColor Green

$gitignoreContent = @"
# Simulation files
sim/*.vvp
sim/*.vcd
sim/*.fst

# Compiler outputs
*.out
*.log

# OS files
.DS_Store
Thumbs.db
```

```
# Editor files
.vscode/.browse.*
*.swp
*~
"@

Set-Content -Path $gitignoreFile -Value $gitignoreContent -Encoding UTF8
```

之后在终端中导航到目录后直接输入:

```
.\create_verilog_project.ps1 -ProjectName "your_project_name" -ModuleName
"your_module_name"
```

(非必要) 批量创建多个项目

创建一个批处理脚本 `batch_create.ps1`:

```
$projects = @(
    @{Name="uart_tx"; Module="uart_transmitter"},
    @{Name="spi_master"; Module="spi_master"},
    @{Name="i2c_controller"; Module="i2c_ctrl"}
)

foreach ($proj in $projects) {
    .\create_verilog_project.ps1 -ProjectName $proj.Name -ModuleName $proj.Module
}

Write-Host "✅ 批量创建完成! " -ForegroundColor Green
```

常见安全策略问题

方法 1: 修改执行策略（推荐，一劳永逸）

以管理员身份打开 PowerShell:

1. 按 `Win + X`，选择 **"Windows PowerShell (管理员)"** 或 **"终端(管理员)"**

2. 运行以下命令：

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

1. 提示时输入 `Y` 确认
2. 关闭管理员窗口，回到普通 PowerShell 重新运行脚本

解释：

- `RemoteSigned`：允许运行本地脚本，网络下载的脚本需要签名
- `CurrentUser`：只对当前用户生效，不影响其他用户

方法 2: 临时绕过（单次运行）

不想改全局策略？可以用这个命令单次运行：

```
PowerShell -ExecutionPolicy Bypass -File .\create_verilog_project.ps1 -ProjectName
"test_project" -ModuleName "test_module"
```

或者在当前会话临时允许：

```
Set-ExecutionPolicy Bypass -Scope Process
.\create_verilog_project.ps1 -ProjectName "test_project" -ModuleName "test_module"
```

方法 3: 解除文件锁定（如果是从网上下载的）

如果你是从网上复制的脚本，Windows 会标记为"不安全"。

解除方法：

1. 右键点击 `create_verilog_project.ps1`
2. 选择 **"属性"**
3. 在底部勾选 **"解除锁定(Unblock)"**
4. 点击 **"确定"**

或者用命令：

```
Unblock-File -Path .\create_verilog_project.ps1
```

然后重新运行脚本。

自动化一键运行脚本

一键运行脚本

`run_verilog.ps1`

保存到项目根目录：

```
# ========================================
# Verilog 一键运行脚本
# 作者: Stomatra
# 功能: 自动编译、仿真、查看波形
# 使用: .\run_verilog.ps1 [testbench文件路径]
# ========================================

param(
    [Parameter(Mandatory=$false, HelpMessage="Testbench 文件路径")]
    [string]$TestbenchFile = "",

    [Parameter(Mandatory=$false, HelpMessage="是否自动打开波形")]
    [switch]$NoWaveform = $false,

    [Parameter(Mandatory=$false, HelpMessage="是否清理旧文件")]
    [switch]$Clean = $false
)

# ========================================
# 配置
# ========================================
$RTL_DIR = "rtl"
$TB_DIR = "tb"
$SIM_DIR = "sim"
```

```
# ========================================
# 函数：打印彩色信息
# ========================================
function Write-Step {
    param([string]$Message, [string]$Color = "Cyan")
    Write-Host ""
    Write-Host "────────────────────────────────────────" -ForegroundColor $Color
    Write-Host "  $Message" -ForegroundColor $Color
    Write-Host "────────────────────────────────────────" -ForegroundColor $Color
}

function Write-Success {
    param([string]$Message)
    Write-Host "✅ $Message" -ForegroundColor Green
}

function Write-Error-Custom {
    param([string]$Message)
    Write-Host "❌ $Message" -ForegroundColor Red
}

function Write-Info {
    param([string]$Message)
    Write-Host "ℹ️  $Message" -ForegroundColor Yellow
}

# ========================================
# 1. 清理旧文件（可选）
# ========================================
if ($Clean) {
    Write-Step "清理仿真文件" "Yellow"
    if (Test-Path $SIM_DIR) {
        Remove-Item -Path "$SIM_DIR\*" -Force -ErrorAction SilentlyContinue
        Write-Success "清理完成"
    }
}

# ========================================
```

```
# 2. 确保 sim 目录存在
# ========================================
if (-not (Test-Path $SIM_DIR)) {
    New-Item -ItemType Directory -Path $SIM_DIR | Out-Null
    Write-Info "创建 sim 目录"
}

# ========================================
# 3. 查找 Testbench 文件
# ========================================
Write-Step "查找 Testbench 文件" "Cyan"

if ($TestbenchFile -eq "") {
    # 自动查找 tb 目录下的文件
    $tbFiles = Get-ChildItem -Path $TB_DIR -Filter "*.v" -ErrorAction SilentlyContinue

    if ($tbFiles.Count -eq 0) {
        Write-Error-Custom "未找到 testbench 文件! "
        Write-Host "请确保 tb 目录下有 .v 文件" -ForegroundColor Gray
        exit 1
    }

    if ($tbFiles.Count -eq 1) {
        # 只有一个文件，自动选择
        $TestbenchFile = $tbFiles[0].FullName
        Write-Success "自动选择: $($tbFiles[0].Name)"
    } else {
        # 多个文件，让用户选择
        Write-Host "找到多个 testbench 文件:" -ForegroundColor Yellow
        for ($i = 0; $i -lt $tbFiles.Count; $i++) {
            Write-Host "  [$i] $($tbFiles[$i].Name)" -ForegroundColor White
        }

        $selection = Read-Host "请选择 [0-$($tbFiles.Count - 1)]"

        if ($selection -match '^\d+$' -and [int]$selection -lt $tbFiles.Count) {
            $TestbenchFile = $tbFiles[[int]$selection].FullName
            Write-Success "已选择: $($tbFiles[[int]$selection].Name)"
```

```
        } else {
            Write-Error-Custom "无效选择！"
            exit 1
        }
    }
} else {
    # 用户指定了文件
    if (-not (Test-Path $TestbenchFile)) {
        Write-Error-Custom "文件不存在: $TestbenchFile"
        exit 1
    }
}

# 获取文件名（不含扩展名）
$tbBaseName = [System.IO.Path]::GetFileNameWithoutExtension($TestbenchFile)
$vvpFile = "$SIM_DIR\$tbBaseName.vvp"
$vcdFile = "$SIM_DIR\$tbBaseName.vcd"

Write-Host "   Testbench: $tbBaseName.v" -ForegroundColor Gray
Write-Host "   输出文件: $tbBaseName.vvp" -ForegroundColor Gray

# ========================================
# 4. 编译 Verilog
# ========================================
Write-Step "编译 Verilog 代码" "Cyan"

# 检查 RTL 目录
if (-not (Test-Path $RTL_DIR)) {
    Write-Error-Custom "未找到 rtl 目录！"
    exit 1
}

# 获取所有 RTL 文件
$rtlFiles = Get-ChildItem -Path $RTL_DIR -Filter "*.v" -Recurse | Select-Object -
ExpandProperty FullName

if ($rtlFiles.Count -eq 0) {
    Write-Error-Custom "rtl 目录下没有 .v 文件！"
```

```
    exit 1
}

Write-Info "找到 $($rtlFiles.Count) 个 RTL 文件"

# 构建编译命令
$compileArgs = @(
    "-g2012",           # SystemVerilog-2012 标准
    "-Wall",            # 显示所有警告
    "-o", $vvpFile,     # 输出文件
    "-I", $RTL_DIR      # 包含路径
)

# 添加所有 RTL 文件
$compileArgs += $rtlFiles

# 添加 testbench 文件
$compileArgs += $TestbenchFile

Write-Host "   执行: iverilog $($compileArgs -join ' ')" -ForegroundColor Gray

try {
    $output = & iverilog $compileArgs 2>&1

    if ($LASTEXITCODE -ne 0) {
        Write-Error-Custom "编译失败！"
        Write-Host $output -ForegroundColor Red
        exit 1
    }

    # 检查警告
    if ($output) {
        Write-Host $output -ForegroundColor Yellow
    }

    Write-Success "编译成功！"

} catch {
```

```
    Write-Error-Custom "编译出错: $_"
    exit 1
}

# ========================================
# 5. 运行仿真
# ========================================
Write-Step "运行仿真" "Cyan"

try {
    Write-Host "   执行: vvp $vvpFile" -ForegroundColor Gray

    $simOutput = & vvp $vvpFile 2>&1

    if ($LASTEXITCODE -ne 0) {
        Write-Error-Custom "仿真失败! "
        Write-Host $simOutput -ForegroundColor Red
        exit 1
    }

    Write-Host ""
    Write-Host "────────────────────────────────────────" -ForegroundColor White
    Write-Host $simOutput
    Write-Host "────────────────────────────────────────" -ForegroundColor White
    Write-Host ""

    Write-Success "仿真完成! "

} catch {
    Write-Error-Custom "仿真出错: $_"
    exit 1
}

# ========================================
# 6. 查看波形
# ========================================
if (-not $NoWaveform) {
    Write-Step "查看波形" "Cyan"
```

```
    if (Test-Path $vcdFile) {
        Write-Info "波形文件: $vcdFile"

        # 检查 GTKWave 是否安装
        $gtkwaveExists = Get-Command gtkwave -ErrorAction SilentlyContinue

        if ($gtkwaveExists) {
            Write-Host "   启动 GTKWave..." -ForegroundColor Gray
            Start-Process gtkwave -ArgumentList $vcdFile
            Write-Success "GTKWave 已启动"
        } else {
            Write-Error-Custom "未找到 GTKWave! "
            Write-Host "请安装 GTKWave 或使用 -NoWaveform 参数跳过" -ForegroundColor
Yellow
        }
    } else {
        Write-Info "未找到波形文件 (.vcd)"
        Write-Host "可能是 testbench 中没有调用 `$dumpfile" -ForegroundColor Gray
    }
}

# ========================================
# 7. 完成
# ========================================
Write-Host ""
Write-Host "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━" -ForegroundColor Green
Write-Host "  ✨ 运行完成! " -ForegroundColor Green
Write-Host "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━" -ForegroundColor Green
Write-Host ""
Write-Host "生成的文件:" -ForegroundColor White
Write-Host "   📄 $vvpFile" -ForegroundColor Gray
if (Test-Path $vcdFile) {
    Write-Host "   📊 $vcdFile" -ForegroundColor Gray
}
Write-Host ""
```

使用方法

方法1:自动模式（最简单）：

在项目根目录直接运行：

```
.\run_verilog.ps1
```

脚本会：

1. 自动找到 `tb/` 下的测试文件
2. 如果只有一个，自动运行
3. 如果有多个，让你选择

方法 2: 指定文件

```
.\run_verilog.ps1 -TestbenchFile "tb\tb_counter.v"
```

方法 3: 不打开波形

```
.\run_verilog.ps1 -NoWaveform
```

方法 4: 清理后运行

```
.\run_verilog.ps1 -Clean
```

方法 5: 组合参数

```
.\run_verilog.ps1 -TestbenchFile "tb\tb_uart.v" -Clean -NoWaveform
```